Android Sale Item Category Service Guide

The Merchant Sale Category Service provides all sale category-related operations—such as retrieval, creation, updating, and deletion of sale category data.

Below are the key utility features available:

- **Create Merchant Sale Categories**: Creates one or more sale categories under the merchant's account.
- **Get All/Specific Merchant Sale Categories**: Processes the Get Sale Item Category request and retrieves the corresponding categories for the merchant. This can be used to obtain details for a specific category or to fetch the complete list of categories associated with the requested user.
- **Update Existing Merchant Sale Categories**: Processes the Update Merchant Sale Category request and updates the specified sale categories for the merchant.
- **Delete Merchant Sale Categories**: Processes the Delete Sale Item Category request and removes the specified sale category for the merchant.

Prerequisites

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and

ON THIS PAGE

Prerequisites

Create Merchant Sale Categories

Get All/Specific Merchant Sale Categories

Update Existing Merchant Sale Categories

Delete Merchant Sale Categories

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Create Merchant Sale Categories

Creates one or more sale categories under the merchant's account.

Step 1: Create a SaleItemCategoryService Object

```
var saleItemCategoryService =
SaleItemCategoryService()
```

Step 2: Create Merchant Sale Category

The CreateSaleItemCategoryRequest is used to create new sale categories for a merchant account. It includes accessToken (via MerchantAccountService) and a categoryList of MerchantSaleItemCategory objects with details like categoryId and categoryName.

```
var saleItemCategory = MerchantSaleItemCategory(
categoryId: 101,
categoryName: "Beverages"
)

var merchantSaleItemCategoryRequest =
CreateSaleItemCategoryRequest(
accessToken: token,
categoryList: categoryList
);
```

Pass the request and bearer token to the createSaleItemCategory() function.

```
 var response =
saleItemCategoryService.createSaleItemCategory(
     createSaleItemCategoryRequest =
merchantSaleItemCategoryRequest,
     bearerToken = token
 )
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

- **If success = true**, response will contain A list of MerchantSaleItemCategory objects, each containing details such as categoryId and categoryName.
- **If success = false**, indicates failure and corresponding failures reasons can be found in the errorCode and message fields.

Get All/Specific Merchant Sale Categories

Processes the Get Sale Item Category request and retrieves the corresponding categories for the merchant. This can be used to obtain details for a specific category or to fetch the complete list of categories associated with the requested user.

Step 1: Create a SaleItemCategoryService Object

```
var saleItemCategoryService =
SaleItemCategoryService()
```

Step 2: Get Merchant Sale Item category

- **accessToken**: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.
- **categoryID**: ID of a specific sale item category to retrieve, or null to fetch all.
- **page**: The page number for pagination.
- **pageSize**: The number of items to fetch per page.

```
var saleItemCategoryRequest =
GetSaleItemCategoryRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
categoryId: "category-456",
page: 1,
pageSize: 10
);
```

Note: *If there is a valid categoryId given in the request, then only that specific sale item category details will be fetched, otherwise it will fetch the entire sale item categories for the requested merchant.*

```
 var response =
saleItemCategoryService.getSaleItemCategory(
     getMerchantSaleItemRequest =
saleItemCategoryRequest,
     bearerToken = token
 )
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

The getSaleItemCategory() function returns a GetSaleItemCategoryResponse object which contains the following:

- **If success = true**, the response will contain list of MerchantSaleItemCategory objects, each containing details such as categoryId and categoryName.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

Update Existing Merchant Sale Categories

Step 1: Create a SaleItemCategoryService Object

```
var saleItemCategoryService =
SaleItemCategoryService()
```

Step 2: Update Merchant Sale Item category

```
var updateSaleItemCategoryRequest =
UpdateSaleItemCategoryRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
categoryList: categorylist
);
```

Pass the request and bearer token to the updateSaleItemCategory() function.

```
 var response =
saleItemCategoryService.updateSaleItemCategory(
     updateSaleItemCategoryRequest =
updateSaleItemCategoryRequest,
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

The updateSaleItemCategory() function returns a UpdateSaleItemCategoryResponse object which contains the following:

- **If success = true**, response will contain list of updated MerchantSaleItemCategory objects, each containing details such as categoryId and categoryName.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

Delete Merchant Sale Categories

Processes the Delete Sale Item Category request and removes the specified sale category for the merchant.

Step 1: Create a SaleItemCategoryService Object

```
var saleItemCategoryService =
SaleItemCategoryService()
```

Step 2: Delete a Merchant Sale Item Category

The DeleteSaleItemCategoryRequest includes accessToken (via MerchantAccountService) and a categoryList of MerchantSaleItemCategory objects with details like categoryId and categoryName to be deleted.

```
var deleteSaleItemCategoryRequest =
DeleteSaleItemCategoryRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
categoryList: categoryList
);
```

Pass the request and bearer token to the deleteSaleItemCategory() function.

```
 var response =
saleItemCategoryService.deleteSaleItemCategory(
     deleteSaleItemCategoryRequest =
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

The deleteSaleItemCategory() function returns a DeleteSaleItemCategoryResponse object which contains the following:

- **If success = true**, response will contain list of MerchantSaleItemCategory object which is deleted.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

Note: *Please refer to the Complete SDK Reference for additional details Android SDK Reference Guide*

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation